Politechnika Wrocławska

Struktury danych		
Temat:	Projekt 1 — Kolejki priorytetowe, porównanie trzech implementacji	
Data:	27.04.2026	**Grupa:** Wtorek 15:15
Autorzy:	Jakub Ryżewski (280047) Dominik Zając (284157)	

Prowadzący:

mgr inż. Piotr Nowak

Spis treści

1 Wstęp **3**

2 Implementacja **3**

3 Metodyka **3**

4 Wyniki **4**

4.1 Operacja `push` 4

4.2 Operacja `pop` 4

4.3 Operacja `peek` 5

4.4 Operacja `changePriority` 6

5 Wnioski **7**

1 Wstęp

Zaimplementowano kolejkę priorytetową w języku C++ na trzech strukturach i porównano czas wykonywania pięciu operacji (`push`, `pop`, `peek`, `getSize`, `changePriority`):

- **Kopiec binarny** (max-heap) z mapą haszującą `id` → indeks,
- **Tablica nieposortowana** (`std::vector`),
- **Lista jednokierunkowa** posortowana malejąco (max w głowie).

Operacja `getSize` jest we wszystkich implementacjach stałoczasowa (odczyt jednego pola) i nie była mierzona osobno.

2 Implementacja

Operacja	**Kopiec**	**Tablica**	**Lista**
`push`	$O(\log N)$	$O(1)^*$	$O(N)$
`pop`	$O(\log N)$	$O(N)$	$O(1)$
`peek`	$O(1)$	$O(N)$	$O(1)$
`changePriority`	$O(\log N)$	$O(N)$	$O(N)$

Tabela 1: Teoretyczne złożoności (przypadek średni). *`push` do tablicy jest zamortyzowane stałoczasowe.

W kopcu mapa `std::unordered_map<int,int>` pozwala znaleźć element po `id` w czasie oczekiwanym $O(1)$, dzięki czemu `changePriority` sprowadza się do `sift-up`/`sift-down` ($O(\log N)$). W tablicy i liście wyszukiwanie po `id` jest liniowe.

3 Metodyka

Pomiary wykonano na Windows 11, kompilator `g++` (MinGW-W64 15.2), `std::chrono::high_resolution_clock`

Procedura. Kolejka jest wypełniana N elementami, włączany jest stoper, mierzona operacja wykonuje się 1000 razy, stoper jest zatrzymywany, a uzyskany czas dzielony przez 1000 — wynik to średni czas pojedynczej operacji w nanosekundach.

Rozmiary. $N \in \{10^3, 10^4, 10^5, 10^6, 10^7\}$ dla wszystkich struktur i operacji. Wyjątek: `push avg` listy posortowanej ($N \leq 10^5$) — wymaga listy z *losowymi* priorytetami, której wypełnienie ma złożoność $O(N^2)$, więc dla $N = 10^7$ samo przygotowanie testu trwałoby ponad 10 godzin.

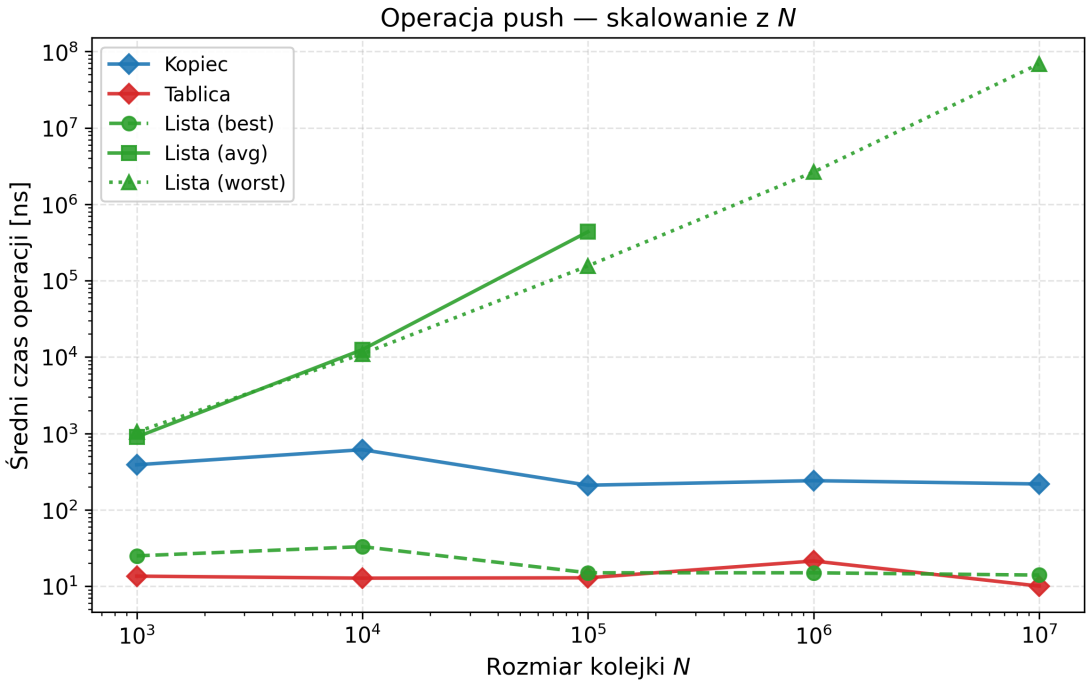
Scenariusze. Dla operacji o niejednorodnej złożoności wyróżniono: **best** — układ minimalizujący koszt, **avg** — losowy, **worst** — wymuszający maksymalny koszt. Dla operacji jednorodnych (np. `push` tablicy, `pop` listy) podawany jest pojedynczy wynik.

4 Wyniki

4.1 Operacja push

N	Kopiec	Tablica	Lista (best)	Lista (avg)	Lista (worst)
10^3	389	13.56	25	898	1 047
10^4	613	12.76	33	12 498	10 989
10^5	210	12.89	15	436 048	155 862
10^6	241	21.41	15	—	2 638 583
10^7	218	10.06	14	—	68 866 862

Tabela 2: Średni czas operacji push [ns].

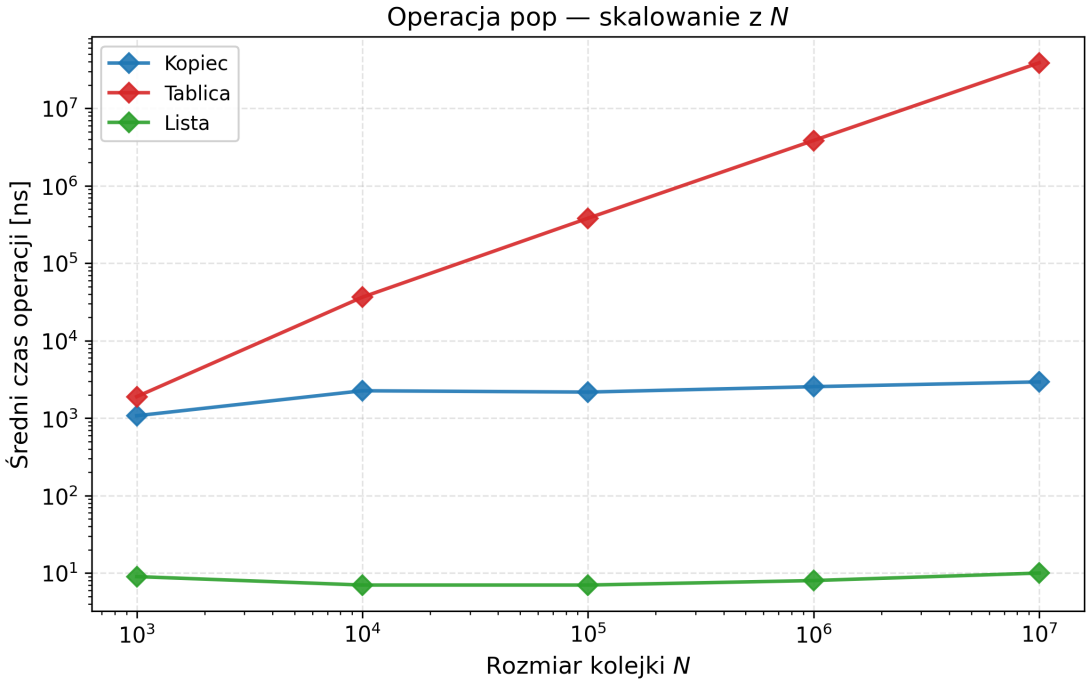


Rysunek 1: push (skala log–log). Tablica stałoczasowa, kopiec logarytmiczny, lista worst/avg liniowa.

4.2 Operacja pop

N	Kopiec	Tablica	Lista
10^3	1 075	1 896	9
10^4	2 258	36 473	7
10^5	2 171	381 343	7
10^6	2 550	3 846 441	8
10^7	2 941	38 733 284	10

Tabela 3: Średni czas operacji pop [ns].

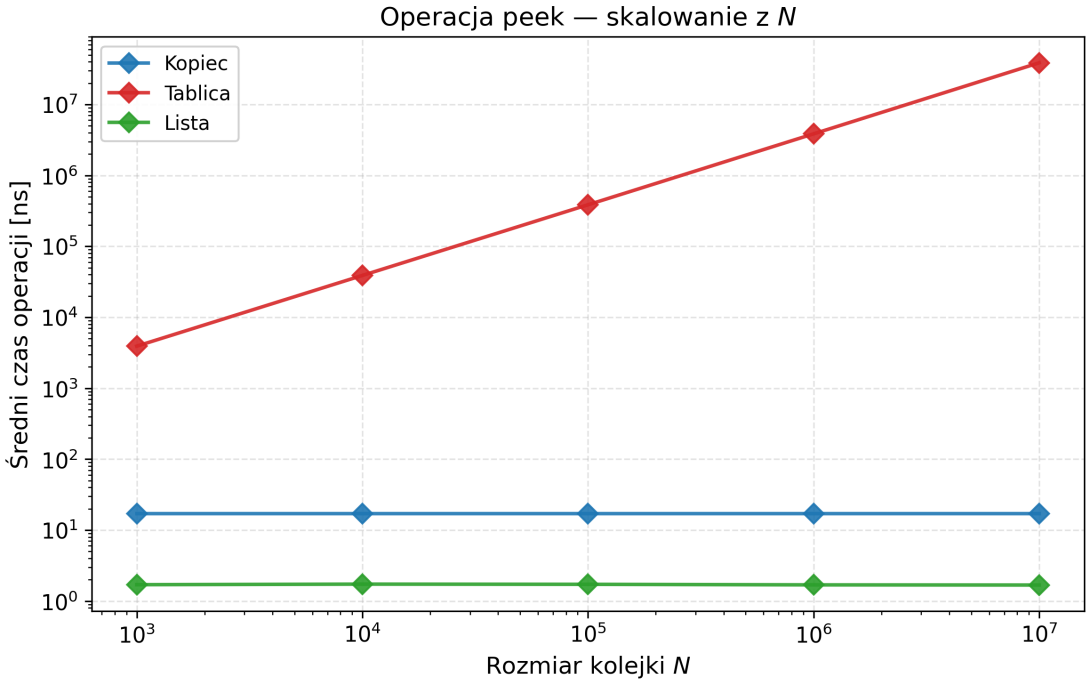


Rysunek 2: `pop`: lista stałoczasowa (usuwa głowę), kopiec logarytmiczny, tablica liniowa (skanowanie).

4.3 Operacja `peek`

N	Kopiec	Tablica	Lista
10^3	17	3 916.24	1.70
10^4	17	38 844.90	1.73
10^5	17	385 104.30	1.72
10^6	17	3 847 948.50	1.69
10^7	17	38 613 165.90	1.68

Tabela 4: Średni czas operacji `peek` [ns].

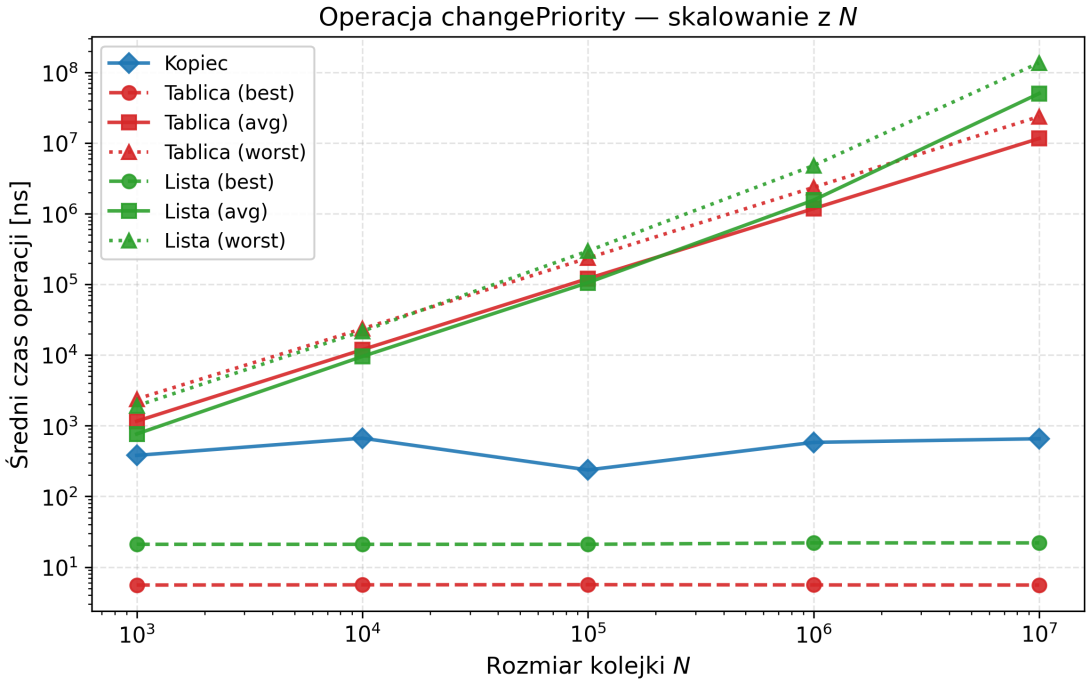


Rysunek 3: `peek`: kopiec i lista zwracają głowę w $O(1)$, tablica wykonuje pełny skan w $O(N)$.

4.4 Operacja `changePriority`

N	Kopiec	Tablica (best)	Tablica (avg)	Tablica (worst)	Lista (best)	Lista (avg)	Lista (worst)
10^3	381	5.57	1 161	2 389	21	765	1 916
10^4	667	5.61	11 919	23 581	21	9 559	21 745
10^5	237	5.64	120 995	236 122	21	105 786	296 800
10^6	582	5.59	1 180 097	2 360 245	22	1 556 124	4 805 638
10^7	655	5.57	11 622 745	23 744 199	22	50 343 836	136 648 525

Tabela 5: Średni czas operacji `changePriority` [ns].



Rysunek 4: `changePriority`: kopiec używa mapy haszującej do znalezienia elementu w $O(1)$ + `sift` $O(\log N)$. Tablica i lista przeszukują liniowo, a wariant `best` obu trafia w element bezpośrednio ($O(1)$).

5 Wnioski

- **Kopiec** jest najbardziej zrównoważony — żadna z operacji nie przekracza $O(\log N)$ i dla dużych N wygrywa w `pop` oraz `changePriority` o kilka rzędów wielkości.
- **Tablica** ma najtańszy `push` ($O(1)$), ale każda operacja wymagająca maksimum (`pop`, `peek`) jest liniowa.
- **Lista** jest dualna do tablicy — `pop` i `peek` stałoczasowe, ale `push`/`changePriority` liniowe w przypadku średnim i pesymistycznym; dodatkowo każdy węzeł wymaga alokacji.
- Nachylenia krzywych w skali log–log pokrywają się z teoretycznymi klasami z tab. 1: linie poziome to $O(1)$, prawie poziome $O(\log N)$, nachylenie 1 to $O(N)$.
- Wybór struktury zależy od profilu operacji: dominacja wstawiania $\rightarrow$ tablica; dominacja `pop`/`peek` $\rightarrow$ lista; mieszany ruch (np. Dijkstra) $\rightarrow$ kopiec.